

基于拥塞势博弈与大语言模型协同的 AI-RAN 计算卸载机制仿真研究

2026 年 7 月 9 日

摘要

现有的 AI-RAN 调度通常假设整个网络是一个有着统一目标的整体（比如由 SMO 全局控盘）。但在未来的异构或多供应商开放网络中，边缘节点和中心节点往往具有冲突的效用函数。比如，Jetson 节点（代表本地边缘）想最大化休眠时间以降低功耗，而 4080 中心（代表运营商）想让 Jetson 保持全功率运转以提升全局 QoS。

研究核心：将 AI-RAN 视为一个经济体，研究大模型 Agent 之间如何基于不完全信息进行纳什均衡谈判。

1 系统模型与理论建模

1.1 网络场景与变量定义

考虑一个包含 1 个中心节点和 N 个边缘节点的 AI-RAN 仿真系统。定义边缘节点集合为 $\mathcal{N} = \{1, 2, \dots, N\}$ 。

每个节点 $i \in \mathcal{N}$ 的卸载策略定义为 $s_i \in \{0, 1\}$ 。

- $s_i = 0$: 任务在边缘节点本地执行。
- $s_i = 1$: 任务通过无线网络卸载至中心节点执行。

系统的全局策略组合表示为 $\mathbf{s} = (s_1, s_2, \dots, s_N)$ 。选择卸载至中心节点的总任务数定义为 $K(\mathbf{s}) = \sum_{j=1}^N s_j$ 。

1.2 极简效用/代价函数 (Cost Function)

我们聚焦于三大硬件工程指标：能耗 (Energy)、算力延迟 (Compute) 与内存约束 (Memory)。边缘节点 i 的总代价函数定义为：

$$C_i(s_i, \mathbf{s}_{-i}) = E_i(s_i) + s_i \cdot (D_{\text{comp}}(K(\mathbf{s})) + M(K(\mathbf{s}))) \quad (1)$$

各项指标具体建模如下：

1. 能耗代价 $E_i(s_i)$:

$$E_i(s_i) = (1 - s_i)e_{i,\text{loc}} + s_i e_{i,\text{tx}}$$

其中 $e_{i,\text{loc}}$ 为本地计算能耗， $e_{i,\text{tx}}$ 为数据传输能耗。

2. **算力拥塞代价** $D_{\text{comp}}(K)$: 中心节点的算力池被建模为 M/M/1 排队系统。拥塞延迟随任务数量 K 非线性递增:

$$D_{\text{comp}}(K) = \frac{1}{\mu - \lambda K}$$

其中 μ 为中心 GPU 处理率, λ 为单个任务请求率。

3. **内存溢出惩罚** $M(K)$: 利用指数障碍函数 (Barrier Function) 模拟物理显存 (如 16GB VRAM) 的硬约束。当总需求逼近显存上限 V_{max} 时, 代价急剧上升:

$$M(K) = \alpha \exp(\beta(K \cdot v_{\text{req}} - V_{\text{max}}))$$

1.3 拥塞势博弈证明 (Potential Game Proof)

定义 1. 如果存在一个全局映射函数 $\Phi: \{0, 1\}^N \rightarrow \mathbb{R}$, 使得任意节点 i 改变策略所引起的自身代价变化, 等于该映射函数的变化量, 则该博弈为精确势博弈 (Exact Potential Game)。

定理 1. 上述算力卸载博弈是一个精确势博弈, 其势函数为:

$$\Phi(\mathbf{s}) = \sum_{i=1}^N E_i(s_i) + \sum_{k=1}^{K(\mathbf{s})} (D_{\text{comp}}(k) + M(k)) \quad (2)$$

证明. 假设仅有节点 i 改变策略从 $0 \rightarrow 1$, 其余节点策略 $\mathbf{s}_{-i}$ 保持不变。此时总卸载数从 K_{-i} 变为 $K_{-i} + 1$ 。节点 i 的代价变化为:

$$\Delta C_i = C_i(1, \mathbf{s}_{-i}) - C_i(0, \mathbf{s}_{-i}) = e_{i,\text{tx}} - e_{i,\text{loc}} + D_{\text{comp}}(K_{-i} + 1) + M(K_{-i} + 1)$$

全局势函数的变化为:

$$\begin{aligned} \Delta \Phi &= \Phi(1, \mathbf{s}_{-i}) - \Phi(0, \mathbf{s}_{-i}) \\ &= (e_{i,\text{tx}} - e_{i,\text{loc}}) + \sum_{k=1}^{K_{-i}+1} (D + M) - \sum_{k=1}^{K_{-i}} (D + M) \\ &= e_{i,\text{tx}} - e_{i,\text{loc}} + D_{\text{comp}}(K_{-i} + 1) + M(K_{-i} + 1) \end{aligned}$$

由于 $\Delta C_i \equiv \Delta \Phi$, 定理得证。该性质在数学上保证了系统通过最佳响应更新, 必然在有限步内收敛至纯策略纳什均衡 (PSNE)。□

2 算法实现：基于 LLM 的语义协调机制

在传统仿真中, 节点需通过全局广播获取准确的 K 值以计算最佳响应。在本文设计中, 中心 LLM 作为语义协调者 (Game Master), 通过“提议-反馈”机制引导博弈收敛, 隐蔽了底层的具体拥塞参数, 增强了网络隐私与弹性。

Algorithm 1 基于 LLM 协同的最佳响应卸载算法 (Simulation)**Require:** 边缘节点集 $\mathcal{N}$, 中心 LLM 协调器, 最大迭代次数 T_{max} **Ensure:** 系统纳什均衡策略 $\mathbf{s}^*$

- 1: **初始化:** 随机生成初始策略 $\mathbf{s}^{(0)} = \{s_1, s_2, \dots, s_N\}$
- 2: **for** $t = 1$ **to** T_{max} **do**
- 3: 随机抽取一个边缘节点 $i \in \mathcal{N}$
- 4: 节点 i 向中心 LLM 发送状态意图: $Intent = \{e_{i,loc}, e_{i,tx}\}$
- 5: 中心 LLM 感知当前全局拥塞状态 $K(\mathbf{s}^{(t-1)})$
- 6: 中心 LLM 模拟节点 i 若执行卸载带来的中心代价增量:

$$\Delta_{center} = D_{comp}(K + 1) + M(K + 1)$$

- 7: LLM 返回语义及数值反馈: $Reply = \{\Delta_{center}, \text{Semantic Warning}\}$
- 8: 节点 i 计算策略翻转代价:
- 9: **if** $e_{i,tx} + \Delta_{center} < e_{i,loc}$ **then**
- 10: $s_i^{(t)} \leftarrow 1$ (决定卸载)
- 11: **else**
- 12: $s_i^{(t)} \leftarrow 0$ (本地执行)
- 13: **end if**
- 14: **if** 连续 N 轮未发生策略改变 **then**
- 15: **Break** (达到纳什均衡)
- 16: **end if**
- 17: **end for**
- 18: **return** $\mathbf{s}^* = \mathbf{s}^{(t)}$

3 仿真测试与验证指标设计

本部分针对纯软件仿真环境（如基于 Python/SimPy），设计以下核心验证指标，以全面评估该理论模型与 LLM 协同框架的性能。

3.1 基准算法对比 (Baselines)

为体现所提算法的优越性，仿真需复现以下基准：

1. **All-Local (全本地):** 所有任务由 Jetson 模拟节点执行（极易耗尽电量，本地算力溢出）。
2. **All-Offload (全卸载):** 所有任务涌向中心（必将导致 4080 显存溢出，触发极高的排队惩罚）。
3. **Random Allocation (随机卸载):** 各节点以 0.5 的概率随机选择。
4. **Greedy Heuristic (传统贪心算法):** 忽略全局拥塞，仅根据局部能耗选择最优。

3.2 核心评估指标 (Evaluation Metrics)

指标一：博弈收敛性分析 (Convergence & Stability)

- **测试方法:** 记录算法每一轮迭代 (Iteration) 后的势函数值 $\Phi(\mathbf{s})$, 绘制收敛曲线。
- **预期结果:** Φ 呈严格单调递减趋势, 并在 N 到 $3N$ 个迭代步长内趋于平缓 (证明有限步收敛定理)。

指标二: 帕累托前沿与系统总代价 (System Total Cost)

- **测试方法:** 在纳什均衡状态下, 统计所有边缘节点的平均能耗代价 (mJ/task) 与算力排队延迟 (ms/task)。
- **预期结果:** 本文算法能自适应地在“降低边缘能耗”与“避免中心算力/内存拥塞”之间找到纳什均衡点, 其总体代价 C_{total} 显著低于所有基准算法。

指标三: 内存硬约束违规率 (Memory Violation Rate)

- **测试方法:** 逐渐增加总节点数 N (从 30 增至 200), 统计 4080 模拟器发生 “Out of Memory” 的频率。
- **预期结果:** 由于障碍函数 $M(K)$ 的存在, 当卸载总需求逼近 VRAM 极限时, LLM 返回极大惩罚, 使违规率严格控制在 0%, 而 Greedy 算法的违规率将大幅上升。

指标四: LLM 协调开销分析 (LLM Orchestration Overhead)

- **测试方法:** 记录仿真过程中, 中心 LLM 作为 “裁判” 产生的 Token 处理量与语义推理时延。
- **预期结果:** 论证虽然引入了 LLM 的推理开销, 但通过将大模型作为 “宏观博弈协调者” 而非 “微观逐包路由者”, 控制信令开销在微秒至毫秒级 (属于 Non-RT RIC 可容忍范围), 具有工程可行性。